

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_09101709-554\agent-a78b8ffc8dade7dac.jsonl`  
- SHA-256: `8848a5923aa0e9bc4d62461a7c8e6aa55e536d34786b6d1a28c27036157d9a7e`  
- Source modified: `2026-06-25T17:12:59+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_09101709-554`  
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`
```

Transcript

1. user (2026-06-25T17:12:32.293Z)

Adversarially verify this PR-review finding for PR #381 (branch chore/nightly-quality-regression checked out; read .github/workflows/nightly.yml + ci.yml). Be skeptical ? default to INVALID if wrong, already-handled, or a non-issue.

Finding [NIT] golden-fixtures lane omits the cross-OS matrix vs its ci.yml source ? intentional and documented, no action needed

Location: .github/workflows/nightly.yml:92-105 vs .github/workflows/ci.yml:120-140

Detail: nightly's `golden-fixtures` runs single-OS (ubuntu-latest) whereas its ci.yml source `golden-crossos` runs a 3-OS matrix (ubuntu/windows/macOS, ci.yml:124-126). The commands and deps are otherwise IDENTICAL (`python -m backend.eval.golden\_fixtures --check` then the two pytest files). This divergence is deliberate and clearly justified in the nightly comment (L88-91: 'single-OS here; ci.yml already covers cross-OS on every PR ? nightly is about drift-over-time on the default branch'). Calling it out only so the reviewer knows the single-OS run is a conscious scope choice, not accidental drift.

Suggestion: No change needed. The comment already explains it.

Return isReal + adjustedSeverity (INVALID if not real) + one-paragraph reasoning grounded in the actual files.

2. assistant (2026-06-25T17:12:34.826Z)

I'll verify this finding by reading the actual workflow files.

3. user (2026-06-25T17:12:36.964Z)

chore/nightly-quality-regression

1a400c0 ci: add Tier-1 nightly quality-regression workflow

86705c1 chore: ruff format gemini_refine.py + trajectory_hybrid.py (#379)

b72ac85 fix(B2/P17): surface Gemini provider failures in all callers (#378)

4. user (2026-06-25T17:12:37.203Z)

```
1      name: Nightly quality regression (Tier-1, offline)  
2  
3      # Tier-1 nightly quality guard. Per-PR `ci.yml` only fires on push/pull_request;  
4      # this runs the SAME offline guards on a schedule so we also catch what per-PR CI  
5      # can't: dependency / transitive-dep drift, flakiness over time, and rot during  
6      # quiet periods when nothing is pushed.  
7      #  
8      # Tier-1 == OFFLINE ONLY: no GPU, no real video, no `run_regression.py` on the  
9      # real corpus (that's Tier-2 ? owner-gated, needs a self-hosted GPU runner +  
10     # the gitignored golden corpus). The lanes here mirror `ci.yml`'s offline jobs  
11     # verbatim (same Python pin, same per-lane pip installs) so modules collect/run  
12     # instead of silently skipping for a missing dep.
```

```

13 #
14 # NOTE: scheduled workflows only fire from the repo's DEFAULT branch, so this
15 # won't actually run on a cadence until it lands on `master`. Use the
16 # `workflow_dispatch` trigger to run it on demand for validation after merge.
17
18 on:
19   schedule:
20     # 03:00 UTC daily.
21     - cron: "0 3 * * *"
22   workflow_dispatch:
23
24 permissions:
25   contents: read
26
27 concurrency:
28   group: nightly-${{ github.ref }}
29   cancel-in-progress: true
30
31 jobs:
32   # Lint gate (ruff.toml at repo root) ? mirrors ci.yml::lint.
33   lint:
34     name: Ruff lint
35     runs-on: ubuntu-latest
36     timeout-minutes: 10
37     steps:
38       - uses: actions/checkout@v6
39       - uses: actions/setup-python@v6
40         with:
41           python-version: "3.13"
42       - name: Install ruff
43         run: python -m pip install ruff
44       - name: Ruff check
45         run: ruff check . --output-format github
46
47   # Type gate (mypy.ini at repo root) ? mirrors ci.yml::typecheck. Typed core
48   # deps are installed so mypy sees real signatures (without them
49   # ignore_missing_imports would Any-stub half the checks).
50   typecheck:
51     name: Mypy (lenient baseline)
52     runs-on: ubuntu-latest
53     timeout-minutes: 10
54     steps:
55       - uses: actions/checkout@v6
56       - uses: actions/setup-python@v6
57         with:
58           python-version: "3.13"
59       - name: Install mypy + typed core deps
60         run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
61 genai numpy
62       - name: Mypy
63         run: mypy backend training
64
65   # Full offline suite + coverage floor ? mirrors ci.yml::full-suite. CPU-only
66   # torch (no CUDA wheels on a CPU runner), then the editable install pulls the
67   # rest. Floor stays at the ci.yml value (70); never lower it without an owner
68   # decision.
69   full-suite:
70     name: Full test suite (offline, mocked)
71     runs-on: ubuntu-latest
72     timeout-minutes: 30
73     steps:
74       - uses: actions/checkout@v6
75       - uses: actions/setup-python@v6
76         with:
77           python-version: "3.13"

```

```

77         cache: pip
78     - name: System libs for opencv-python
79       run: |
80         sudo apt-get update
81         sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev
82     - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
83       run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
84     - name: Install package (editable) + dev tooling
85       run: python -m pip install -e .[dev]
86     - name: Run full suite (with coverage floor)
87       run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
88
89     # Offline quality guard: the committed golden regression fixtures
90     # (docs/GOLDEN_REGRESSION_FIXTURES.md) ? synthetic + de-attributed real
91     # trajectory fixtures, NO video / GPU / DB. Mirrors the offline replay in
92     # ci.yml::golden-crossos (single-OS here; ci.yml already covers cross-OS on
93     # every PR ? nightly is about drift-over-time on the default branch). The
94     # CONTROL replay needs only numpy.
95     golden-fixtures:
96       name: Golden fixtures regression (offline quality guard)
97       runs-on: ubuntu-latest
98       timeout-minutes: 15
99       steps:
100         - uses: actions/checkout@v6
101         - uses: actions/setup-python@v6
102           with:
103             python-version: "3.13"
104         - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
105           run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
106         - name: Golden-fixtures characterization (offline)
107           run: |
108             python -m backend.eval.golden_fixtures --check
109             python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
110
111     # Failure visibility ? a silent nightly is useless. Scheduled runs have no PR
112     # to reddent and nobody is watching the Actions tab on a quiet day (the exact
113     # "rot during quiet periods" this workflow exists to catch). On ANY guard
114     # failure, open/refresh a single tracking issue so a human is pinged. Uses the
115     # built-in GITHUB_TOKEN (no secrets); `issues: write` is scoped to THIS job
116     # only ? the guard jobs above keep `contents: read`.
117     notify-on-failure:
118       name: Surface nightly failure (open/refresh tracking issue)
119       needs: [lint, typecheck, full-suite, golden-fixtures]
120       if: failure() && github.event_name == 'schedule'
121       runs-on: ubuntu-latest
122       timeout-minutes: 5
123       permissions:
124         contents: read
125         issues: write
126       steps:
127         - uses: actions/github-script@v7
128           with:
129             script: |
130               const runUrl =
`${context.serverUrl}/${context.repo.owner}/${context.repo.repo}/actions/runs/${context.runId}`;
131               const title = 'Nightly quality regression FAILED';
132               const body = [
133                 'The Tier-1 nightly offline quality guard failed.',
134                 '',
135                 ` - Run: ${runUrl}`,

```

```

136         '- Date (UTC): ${new Date().toISOString()}`,
137         '',
138         'One of ruff / mypy / full-suite (cov>=70) / golden-fixtures regressed.',
139         'Open the run for the failing lane. Close this issue once green.',
140     ].join('\n');
141     // Reuse a single open tracking issue instead of spamming one per night.
142     const existing = await github.rest.issues.listForRepo({
143         owner: context.repo.owner,
144         repo: context.repo.repo,
145         state: 'open',
146         labels: 'nightly-failure',
147         per_page: 1,
148     });
149     if (existing.data.length > 0) {
150         await github.rest.issues.createComment({
151             owner: context.repo.owner,
152             repo: context.repo.repo,
153             issue_number: existing.data[0].number,
154             body,
155         });
156     } else {
157         await github.rest.issues.create({
158             owner: context.repo.owner,
159             repo: context.repo.repo,
160             title,
161             body,
162             labels: ['nightly-failure'],
163         });
164     }
165

```

5. user (2026-06-25T17:12:37.459Z)

```

1     name: CI
2
3     on:
4       push:
5         branches: [master]
6       pull_request:
7
8     permissions:
9       contents: read
10
11     concurrency:
12       group: ci-${{ github.ref }}
13       cancel-in-progress: true
14
15     jobs:
16       # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17       # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18       # tracked .py, including scripts/ and training/, which the backend-only
19       # py_compile smoke sweep does not reach.
20       lint:
21         name: Ruff lint
22         runs-on: ubuntu-latest
23         steps:
24           - uses: actions/checkout@v6
25           - uses: actions/setup-python@v6
26             with:
27               python-version: "3.13"
28           - name: Install ruff
29             run: python -m pip install ruff
30           - name: Ruff check
31             run: ruff check . --output-format github
32

```

```

33 # PII / attribution leak guard (tools/leak_scan.py): fails on a committed 32-hex MD5
34 # (the video_id shape) or a personal absolute user path in the shipped-code surface
35 # (backend/training/scripts/tools/eval_baselines; tests/ excluded ? legit fixtures).
Real
36 # names are scanned only via a gitignored private list locally; CI runs the structural
guard.
37 leak-scan:
38   name: Leak scan (PII / attribution guard)
39   runs-on: ubuntu-latest
40   steps:
41     - uses: actions/checkout@v6
42     - uses: actions/setup-python@v6
43       with:
44         python-version: "3.13"
45     - name: Leak scan
46       run: python -m tools.leak_scan
47
48 # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
49 # per-module quarantine list ? ratchet by removing entries as modules clean
50 # up. Typed core deps are installed so mypy sees their real signatures
51 # (without them ignore_missing_imports would Any-stub half the checks).
52 typecheck:
53   name: Mypy (lenient baseline)
54   runs-on: ubuntu-latest
55   steps:
56     - uses: actions/checkout@v6
57     - uses: actions/setup-python@v6
58       with:
59         python-version: "3.13"
60     - name: Install mypy + typed core deps
61       run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
62     - name: Mypy
63       run: mypy backend training
64
65 # Fast guard against the 7fbbeb0 class of breakage: a SyntaxError or
66 # module-level import error shipping while the suite stays green.
67 # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
68 # on machines without the GPU stack (test_import_smoke stubs what's absent).
69 import-smoke:
70   name: Syntax sweep + import smoke (no GPU stack)
71   runs-on: ubuntu-latest
72   steps:
73     - uses: actions/checkout@v6
74     - uses: actions/setup-python@v6
75       with:
76         python-version: "3.13"
77     - name: Install minimal deps
78       run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
79     - name: Run import smoke tests
80       run: python -m pytest tests/test_import_smoke.py -v
81
82 full-suite:
83   name: Full test suite (offline, mocked)
84   runs-on: ubuntu-latest
85   steps:
86     - uses: actions/checkout@v6
87     - uses: actions/setup-python@v6
88       with:
89         python-version: "3.13"
90         cache: pip
91     - name: System libs for opencv-python
92       run: |
93         sudo apt-get update

```

```

94         sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev
95     - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
96     # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
97     # need an out-of-band index that PEP 621 can't express. Keep installing
98     # them separately here, before the editable install pulls the rest.
99     run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
100    - name: Install package (editable) + dev tooling
101    # Editable PEP 621 install: runtime deps + the `dev` group
102    # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
103    run: python -m pip install -e .[dev]
104    # obsreport (private, soft dep) is absent here: reporting tests skip via
105    # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
106    # is therefore calibrated against THIS lane's number (local runs with
107    # obsreport installed measure a little higher).
108    # Floor calibrated 2026-06-11 against this lane's measured 72% (local
109    # with obsreport: 73%). Ratchet it UP as coverage grows; never down
110    # without an owner decision.
111    - name: Run full suite (with coverage floor)
112    run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
113
114    # Cross-OS determinism guard for the committed golden regression fixtures
115    # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
116    # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
117    # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
118    # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
119    # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
120    golden-crossos:
121      name: Golden fixtures cross-OS (${{ matrix.os }})
122      strategy:
123        fail-fast: false
124        matrix:
125          os: [ubuntu-latest, windows-latest, macos-latest]
126      runs-on: ${{ matrix.os }}
127      steps:
128        - uses: actions/checkout@v6
129        - uses: actions/setup-python@v6
130        with:
131          python-version: "3.13"
132        - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
133        run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
134    - name: Golden-fixtures characterization (cross-OS determinism guard)
135    # Also runs the freeze-real / time-origin-reset / M2 refusal-gate tests
(synthetic /
136    # tmp_path only ? no GPU/network) so the reset metric-invariance (fix #2) and
the
137    # de-attribution paths get cross-OS coverage and surface any win/mac float
drift.
138    run: |
139      python -m backend.eval.golden_fixtures --check
140      python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
141
142    # Commercial-clean licensing guardrail (MIT/Apache/BSD only ? PACKAGING_PLAN.md P0d).
Scans the
143    # RUNTIME dependency closure (`pip install .` ? no torch/dev) and fails on any
copyleft
144    # (GPL/AGPL/LGPL/SSPL). torch (BSD-3) is installed out-of-band and is not in this
closure. This
145    # automates a guardrail that was human-only; the report is printed for review either
way.
146    license-scan:

```

```

147     name: Dependency license scan (no copyleft in runtime deps)
148     runs-on: ubuntu-latest
149     steps:
150     - uses: actions/checkout@v6
151     - uses: actions/setup-python@v6
152       with:
153         python-version: "3.13"
154     - name: Install runtime deps + pip-licenses
155       run: |
156         python -m pip install .
157         python -m pip install pip-licenses
158     - name: Report dependency licenses
159       run: python -m piplicenses --format=markdown --with-urls --order=license
160     - name: Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
161       run: |
162         if python -m piplicenses --format=plain | grep -iE "GPL|SSPL"; then
163           echo "::error::A runtime dependency carries a copyleft (GPL/AGPL/LGPL/SSPL)
license ? the guardrail is MIT/Apache/BSD only. Review the report above."
164           exit 1
165         fi
166       echo "OK: no GPL/AGPL/LGPL/SSPL in the runtime dependency closure."
167
168     # Built-wheel smoke (PACKAGING_PLAN.md P0d): build the wheel, install it NON-editable
from a
169     # non-repo cwd, and launch the console script. Catches packaging regressions the
source suite
170     # can't ? namespace-package resolution (backend has no __init__.py), missing data
files, hidden
171     # imports. (Validated locally: a clean-venv wheel install + CPU torch runs `indexer
--help`.)
172     build-artifact:
173     name: Built wheel smoke (install + launch)
174     runs-on: ubuntu-latest
175     steps:
176     - uses: actions/checkout@v6
177     - uses: actions/setup-python@v6
178       with:
179         python-version: "3.13"
180     - name: System libs for opencv-python
181       run: |
182         sudo apt-get update
183         sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev
184     - name: Build the wheel
185       run: |
186         python -m pip install build
187         python -m build --wheel
188     - name: Install the built wheel + CPU torch (out-of-band index)
189       run: |
190         python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
191         python -m pip install dist/*.whl
192     - name: Launch the installed console script from a non-repo dir
193       run: |
194         cd /tmp
195         indexer --help
196         python -c "import backend, training, backend.utils.app_home; print('wheel
import OK')"
197

```

6. assistant (2026-06-25T17:12:49.671Z)

Both files are read. The finding is precisely verifiable against the actual source.

Verification of each claim:

1. ****nightly's `golden-fixtures` runs single-OS (ubuntu-latest)**** ? Confirmed. nightly.yml:97 `runs-on: ubuntu-latest`, no matrix.
2. ****ci.yml's `golden-crossos` runs a 3-OS matrix**** ? Confirmed. ci.yml:124-126 `os: [ubuntu-latest, windows-latest, macos-latest]`, ci.yml:126 `runs-on: \${ matrix.os }`.
3. ****Commands and deps otherwise IDENTICAL**** ? Confirmed. Both install `pytest fastapi uvicorn pydantic python-dotenv google-genai numpy` (nightly.yml:105 vs ci.yml:133), and both run `python -m backend.eval.golden\_fixtures --check` then `python -m pytest tests/test\_golden\_fixtures.py tests/test\_golden\_real\_fixtures.py -q` (nightly.yml:108-109 vs ci.yml:139-140).
4. ****The divergence is documented in the nightly comment**** ? Confirmed. nightly.yml:92-93 literally says "single-OS here; ci.yml already covers cross-OS on every PR ? nightly is about drift-over-time on the default branch."

The finding's own conclusion is that this is intentional, justified, documented, and needs no change. By its own admission it's a non-actionable observation. A code-review finding that explicitly recommends "No change needed" and confirms the divergence is a "conscious scope choice, not accidental drift" is not a defect ? it is correct that the behavior is fine, which means there is nothing to fix. Furthermore, the rationale is genuinely sound: scheduled workflows fire only from the default branch (also documented at nightly.yml:14-16), cross-OS determinism is a per-PR concern already fully covered, and nightly's purpose is temporal drift detection where running 3 OSes daily would just triple cost for no added drift signal. There is no real issue here ? the design is correct and self-documenting.

7. user (2026-06-25T17:12:56.006Z)

Structured output provided successfully

8. assistant (2026-06-25T17:12:59.299Z)

The finding is INVALID. I verified all its factual claims against the actual workflow files (`.github/workflows/nightly.yml` and `.github/workflows/ci.yml`), and while they are accurate, the finding itself recommends "No change needed" and confirms the single-OS choice is deliberate and self-documented (nightly.yml:92-93). It is not a real defect ? just a correct observation about sound, documented design.